

SOFTWARE REQUIREMENTS SPECIFICATION (SRS) DOCUMENT

PROJECT NAME: Eventvista

The Technical University of Kenya

Faculty of Social Sciences and Technology

Department of Information Science, Language and Communication Studies

TEAM MEMBERS & ROLES

Lewis Kariuki | AIIM/00477/2021 | Project Manager: Responsible for the overall orchestration of the SDLC. Manages project timelines, oversees system orchestration, coordinates Phase 11 testing protocols, and handles final deployment (Phase 12).

Gideon Oseko | AIIM/02582/2023 | Requirements Gathering & System Analyst: Leads Phase 1 inception, user persona research, and logic mapping. Responsible for validating system boundaries and ensuring user stories accurately reflect the needs of all user types across the platform.

Shekinah Osami Mutta | AIIM/00501/2023 | Lead UI/UX Designer: Leads Phase 2 interface design. Responsible for wireframing the interactive 3D transformation interfaces and designing accessible layout controls. Translates wireframes into the final Figma designs used for the Phase 7 frontend build.

John Simon | AIIM/00510/2023 | Lead Software Developer: Head of technical engineering and pipeline integration. Responsible for executing Phase 8 (Backend APIs), Phase 9 (System Connection), and Phase 10 (3D Venue Designer integration), including the Groq and Hugging Face TRELIS interfacing. John is additionally responsible for building the automated document generation pipeline, which involves integrating MeshLab, LaTeX, and the OpenSign API to produce the final 3D-embedded PDF and manage client e-signatures.

Approval Signatures & Project Endorsement:

Lewis Kariuki Project Manager

Gideon Oseko Lead Requirements Gathering & System Analyst

Shekinah Osami Mutta Lead UI/UX Designer

John Simon Lead Software Developer

Lec Bostone Ochieng Academic Supervisor & Course Lecturer

Tables of content

1. INTRODUCTION

1.1 Purpose

1.2 Problem Statement & Specific Objectives

Strategic Project Objectives:

1.3 Scope

1.4 Value Proposition

1.5 Definitions, Acronyms, and Abbreviations

1.6 References

1.7 Overview

2. GENERAL DESCRIPTION

2.1 Product Perspective

2.2 Product Functions

2.3 User Characteristics

2.4 Constraints

2.5 Assumptions and Dependencies

3. SYSTEM ARCHITECTURE & TECHNICAL SPECIFICATIONS

3.1 System Architecture Component Blueprint

3.2 Backend Logic & Sequence Diagrams

3.3 Database Entity-Relationship Diagram (ERD)

3.4 Core Technology Stack Details

4. FRONTEND INTERFACE REQUIREMENTS

4.1. User Interface Wireframes

4.2 Hardware Interfaces

4.3 Software Interfaces

4.4 Communications Interfaces

5. FUNCTIONAL REQUIREMENTS

[5.1 Frontend Interface Capabilities \(Phase 7 Integration\)](#)

[5.2 Backend API Processing \(Phase 8 Integration\)](#)

[5.3 3D Venue Designer Engine \(Phase 10 Integration\)](#)

[5.2 Interactive Event Configuration Setup \(CRUD Operations\)](#)

[5.3 2D-to-3D Visual Pipeline Processing](#)

[5.4 Groq-Driven Predictive Analytics Engine](#)

[5.5 Guest Management, Ticket Processing, and Automated Communications](#)

[6. NON-FUNCTIONAL REQUIREMENTS](#)

[6.1 Performance and Speed Requirements](#)

[6.2 Data Security and Encryption](#)

[6.3 Scalability and Availability](#)

[7. Project Management & Analysis Models](#)

[7.1 Project Implementation Plan & Methodology \(Agile Framework\)](#)

[Why we chose Agile for Eventvista:](#)

[7.2 Core Work Breakdown Structure \(WBS\) Tasks](#)

[8. LEGAL AND ETHICAL CONSIDERATIONS](#)

[8.2 Ethical and Explainable Use of AI](#)

[8.3 Oversight Bodies](#)

[9. CONCLUSION AND FUTURE RECOMMENDATIONS](#)

[9.2 Future Iterations and Recommendations](#)

[9.3 Project Potential for Research and Teaching](#)

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document defines the comprehensive functional, non-functional, business, and architectural requirements for Eventvista. Eventvista is an AI-driven, end-to-end event-organisation ecosystem built to democratise professional-grade spatial planning and resource forecasting for everyday users.

1.2 Problem Statement & Specific Objectives

Traditional event coordination remains fragmented, expensive, and manual. Casual event planners struggle to balance structural room dimensions, local budget ceilings, and logistical safety calculations. Existing software options (e.g., Cvent, Social Tables) cater exclusively to enterprise venue operators, demanding complex CAD layout uploads or extensive pre-rendered design matrices.

Strategic Project Objectives:

To build an integrated web platform that reduces event setup times by over 70% for everyday users.

To provide a high-fidelity image-to-3D transformation tool capable of converting real-world photos into functional layouts within 15 seconds.

To embed predictive analytical tools that continuously manage the project's "Triple Constraints" (Scope, Time, and Budget).

To create a secure marketplace architecture for instant local component procurement.

1.3 Scope

Eventvista eliminates the fragmentation of traditional event planning by combining administrative event workflows with an advanced generative AI spatial model pipeline. The system takes a standard 2D image of an empty venue captured by a user's mobile device, interprets the room boundaries, maps architectural constraints, processes event metrics (guest numbers, resource availability, budget limits), and synthesises an interactive, customisable 3D event layout canvas.

1.4 Value Proposition

'Eventvista's core value rests on three distinct pillars:

Democratised Spatial Planning: Eliminates spatial layout guesswork. A user can step inside an empty hall, capture a panoramic image, and instantly review how 200 dining chairs, a banquet table array, and a performance stage fit dynamically into the space.

Real-Time Predictive Budget Optimisation: The Groq-powered backend acts as a virtual project consultant, warning users if adding a furniture element violates structural boundaries, safety clearances, or allocated budget metrics.

Frictionless Transaction Execution: Integrating the Ms-Pesa API means African event organisers can coordinate vendor execution directly within localised financial grid.

1.5 Definitions, Acronyms, and Abbreviations

SRS: Software Requirements Specification

LPU: Language Processing Unit (Groq Hardware Architecture)

PBR: Physically Based Rendering (Texture and material mapping)

API: Application Programming Interface

CRUD: Create, Read, Update, Delete

WBS: Work Breakdown Structure

1.6 References

This section lists the primary external technologies, frameworks, and Application Programming Interfaces (APIs) referenced throughout this document. These tools form the foundational technology stack required to build and deploy Eventvista:

Next.js: A robust React-based web development framework used to build 'Eventvista's interactive frontend user interface. It provides fast page rendering and a highly responsive user experience.

Vercel: A modern cloud hosting platform specifically optimised for frontend applications. It automatically deploys the Next.js website, ensuring rapid global access and seamless updates without manual server configuration.

Render: A fully managed cloud hosting service used to power 'Eventvista's backend logic, API gateway, and secure databases. It handles automatic scaling and security, allowing the development team to focus on building features rather than managing complex infrastructure.

Groq API: An ultra-fast Artificial Intelligence (AI) inference engine powered by Language Processing Units (LPUs). Eventvista relies on this API to instantly process user text inputs, calculate event budgets, and perform predictive resource analytics in mere milliseconds.

Hugging Face TRELIS: A state-of-the-art, open-source AI model hosted on the Hugging Face platform. It specialises in generating high-quality 3D assets from standard 2D images, serving as the core engine that powers 'Eventvista's interactive venue designer.

1.7 Overview

The remainder of this Software Requirements Specification provides a comprehensive blueprint for the Eventvista platform. Section 2 offers a general description of the software, highlighting its primary functions, technical constraints, and target audience. Section 3 explores the technical architecture, providing visual diagrams of the backend workflows alongside the project's estimated financial plan. Section 4 maps out the external interfaces, prominently featuring the high-fidelity frontend design wireframes. Section 5 defines the core functional requirements, detailing exactly how the AI spatial engine and event management modules operate. Section 6 establishes strict non-functional standards, such as system security, data privacy, and performance metrics. Finally, Section 7 outlines the project management methodology, detailing the step-by-step Work Breakdown Structure (WBS) used to guide the team through the software development life cycle.

2. GENERAL DESCRIPTION

2.1 Product Perspective

Eventvista operates as a cross-platform web application optimised for desktop customisation and mobile image capturing. The system relies heavily on external high-performance APIs for low-latency computation: the Groq API handles deterministic configuration logic, predictive resource mapping, and user parameter interpretation, while the open-source TRELLIS model pipeline, hosted on Hugging Face, parses visual structural boundaries to deliver interactive 3D assets to the user's browser.

2.2 Product Functions

Automated venue parsing via image upload.

Predictive resource, asset, and budget allocation modelling.

Interactive, browser-based drag-and-drop 3D space modification.

End-to-end event management metrics tracking (guest registration, automated messaging pipelines).

Localised payment orchestration.

2.3 User Characteristics

The target user is an everyday consumer or casual corporate planner with zero technical training in Computer-Aided Design (CAD), spatial engineering, or advanced spreadsheet accounting. The interface prioritises high-accessibility mechanics over clinical design configurations.

2.4 Constraints

This section outlines the technical limitations and boundaries that must be considered during the development and operation of the Eventvista platform. These constraints ensure that the system remains stable and predictable for all users:

Hardware Limitations: Eventvista requires a device with a modern Graphics Processing Unit (GPU) to handle real-time 3D rendering. Users on older hardware or devices without dedicated graphics acceleration may experience reduced performance, such as slower 3D model generation or lower frame rates within the venue designer.

Browser Compatibility: The platform relies on WebGL and React Three Fibre to display 3D environments directly in the web browser. Therefore, users must have a modern, up-to-date web browser (such as Chrome, Edge, Safari, or Firefox) that supports WebGL 2.0. Older browsers or those with strict security settings that block WebGL will be unable to render the 3D canvases.

API Rate Limits: To maintain performance and manage operational costs, Eventvista operates under specific API rate limits. This includes caps on the number of requests sent to the Groq AI engine and the Hugging Face model per user session. Exceeding these limits during a single session will result in temporary service throttling, requiring the user to wait before performing additional AI-driven tasks.

2.5 Assumptions and Dependencies

This section identifies the critical external factors and services that Eventvista relies on to function correctly. The successful operation of the platform is dependent on the availability and performance of these third-party systems:

Dependency on Hugging Face GPU Availability: The ability to convert 2D venue images into 3D models relies on the TRELIS AI model hosted on Hugging Face. Eventvista assumes that these hosted GPU spaces remain available. If the Hugging Face service encounters downtime or high server traffic, the 3D generation feature may experience delays or become temporarily unavailable.

Groq API Response Latency: Eventvista's resource forecasting and budgeting features are powered by the Groq LPU engine. We assume that the Groq API will maintain its standard low-latency performance. Because the system is designed for real-time interaction, any significant spikes in network latency or service degradation from the Groq API provider will directly affect the speed at which users receive budget calculations and planning insights.

Reliable Cloud Infrastructure: It is assumed that our hosting partners, Vercel and Render, will maintain 99.9% uptime. The application depends on these services for constant access to the database and frontend assets; therefore, any unplanned outages in these hosting environments would impact the overall accessibility of the Eventvista platform.

3. SYSTEM ARCHITECTURE & TECHNICAL SPECIFICATIONS

3.1 System Architecture Component Blueprint

Eventvista uses an API Gateway architecture to coordinate distributed computational resources smoothly:

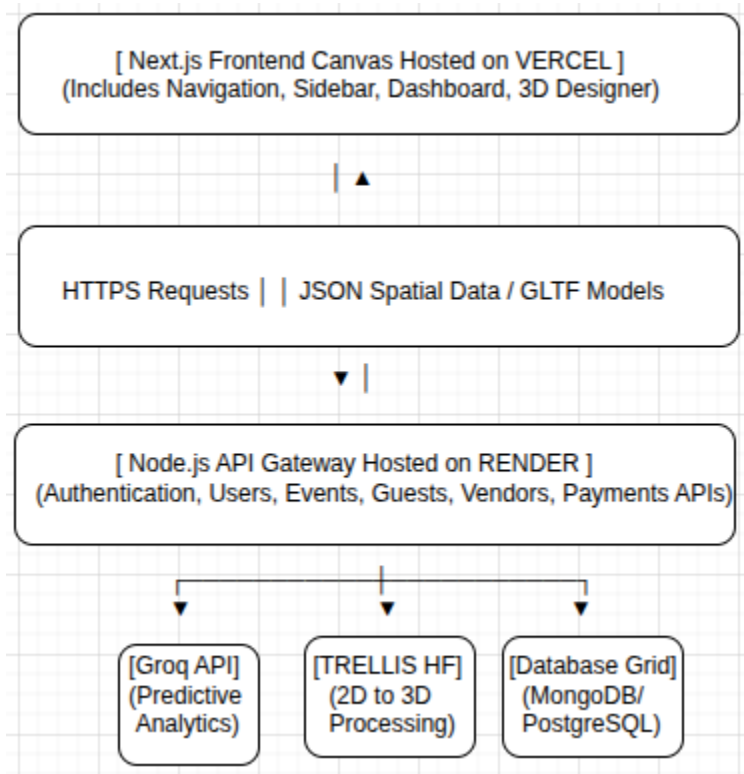
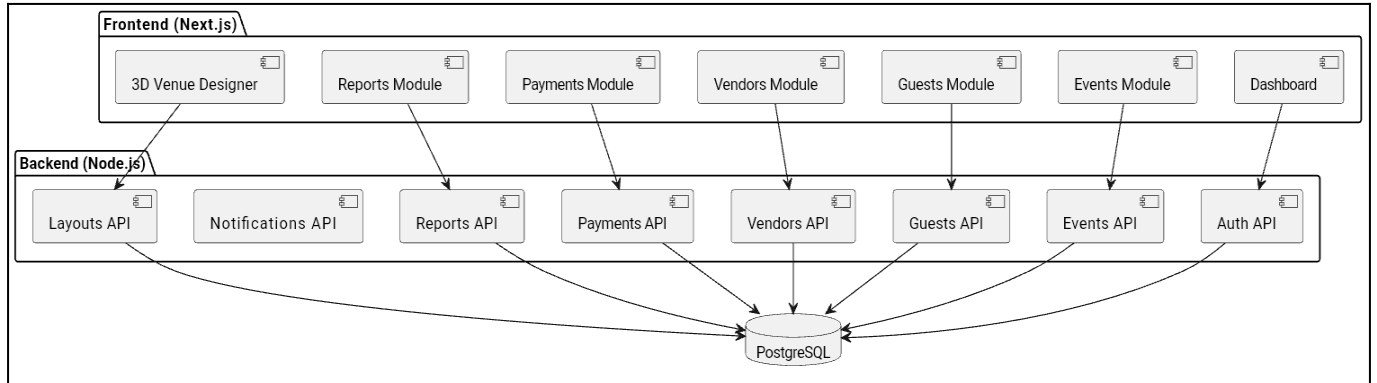
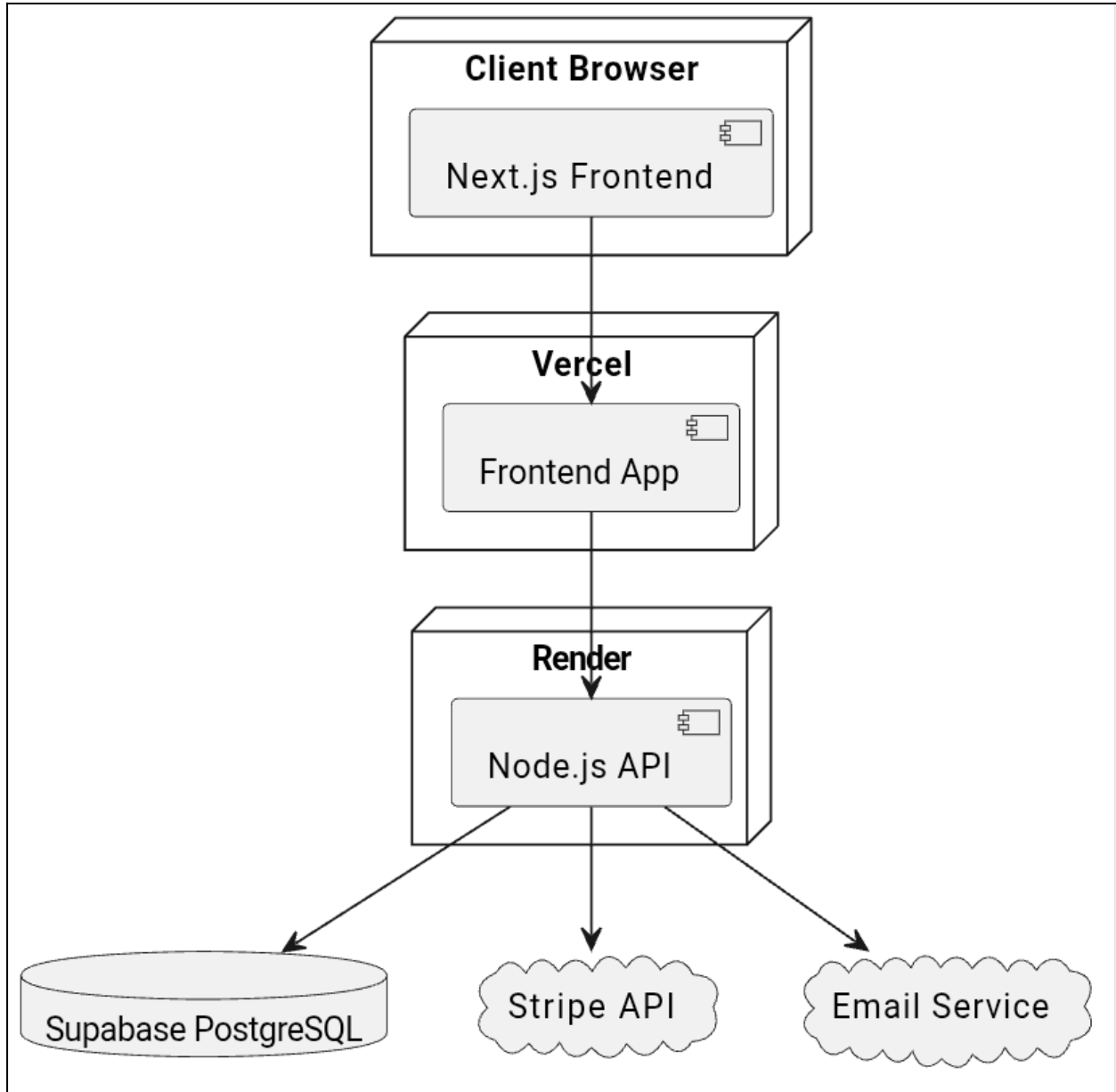


Figure 3.1: Eventvista Component Diagram



This diagram breaks down the software system into its primary structural modules. It illustrates the Next.js client application communicating with the central Node.js API Gateway, which securely controls the flow of data across individual sub-services, database engines, and third-party AI frameworks.

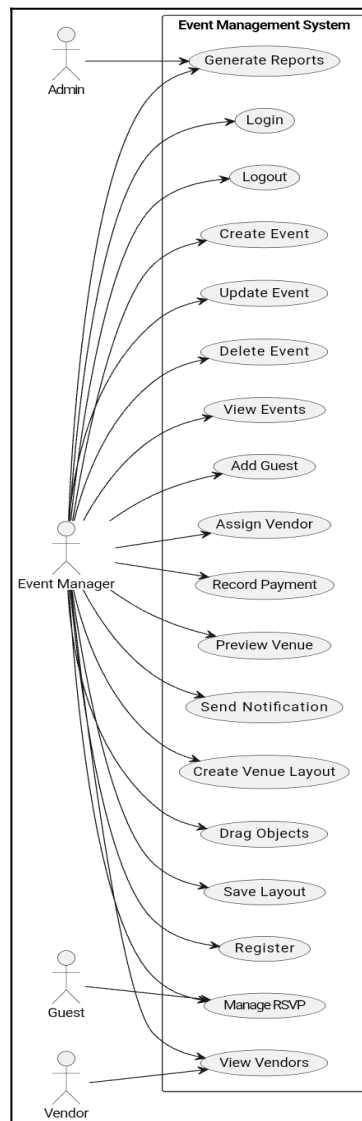
Figure 3.2: Eventvista System Deployment Diagram



This diagram visualises the physical hosting and runtime cloud infrastructure of the platform. It shows how end-users access the frontend user interface deployed on Vercel, which securely establishes network endpoints with the central logic service hosted on Render and communicates with high-performance Hugging Face GPU spaces.

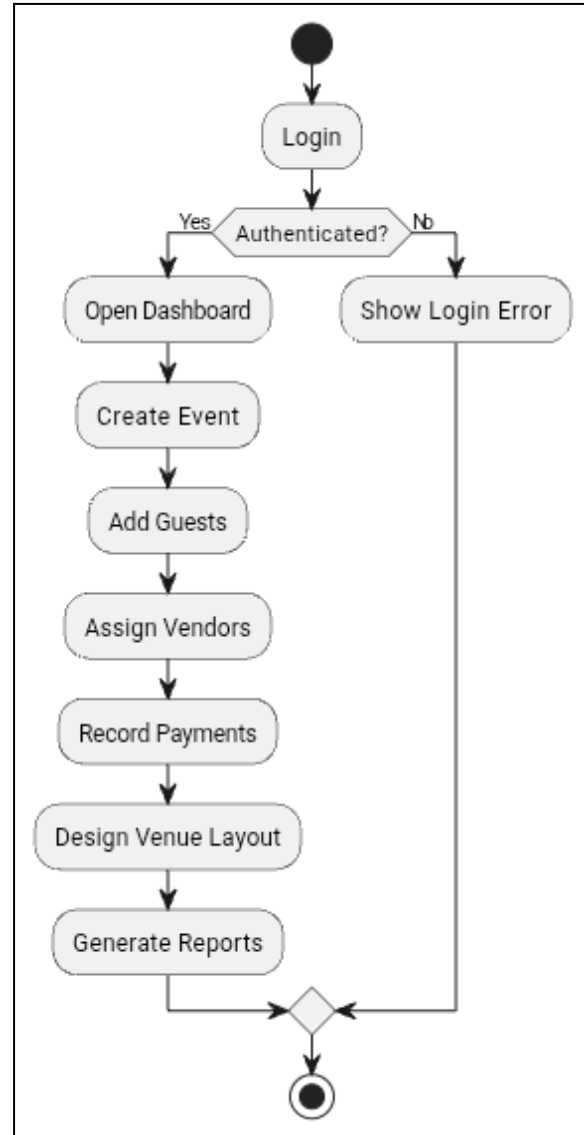
3.2 Backend Logic & Sequence Diagrams

Figure 3.3: Eventvista Use Case Diagram



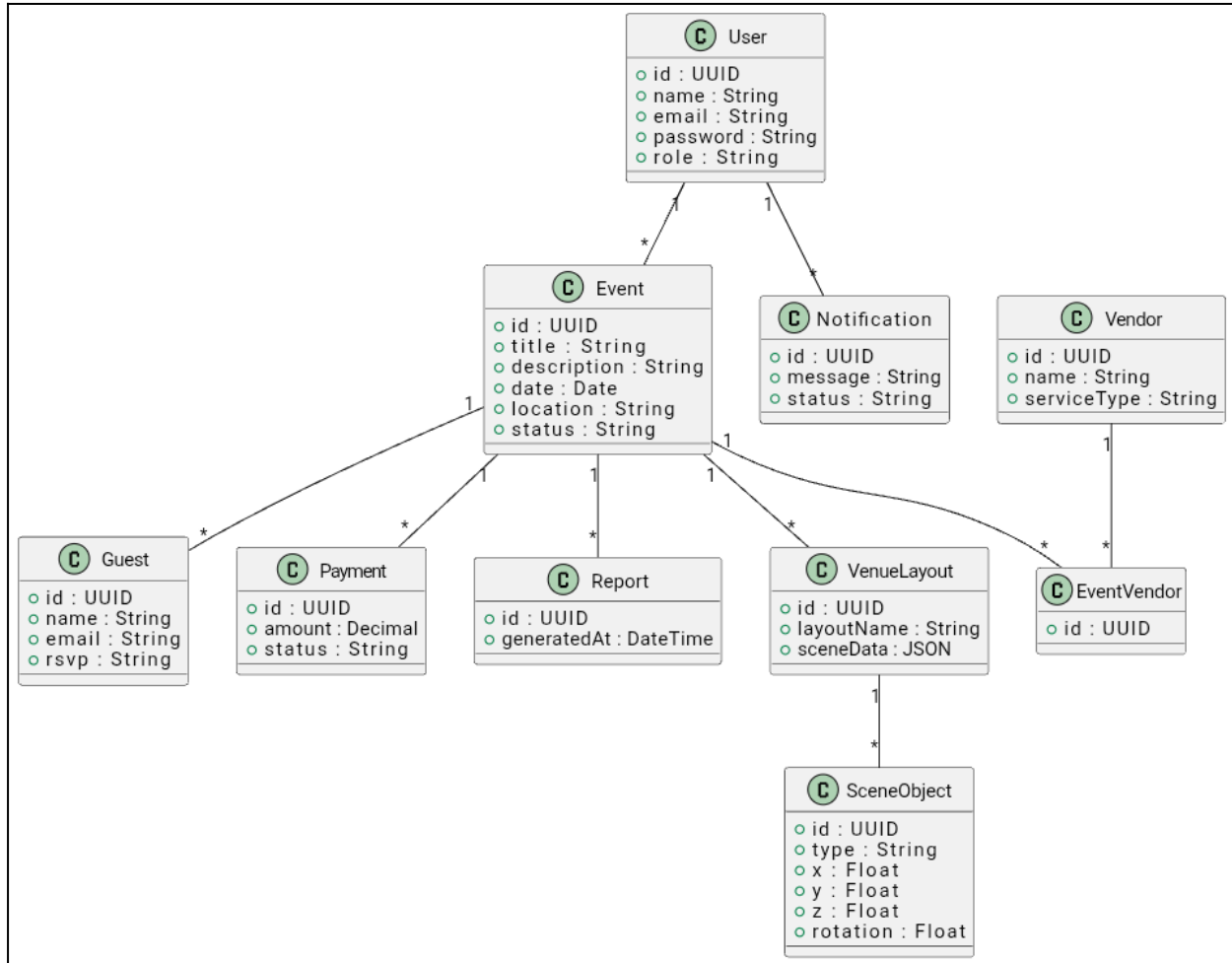
This diagram maps out the interactions between external actors (Organisers, Attendees, and Vendors) and the system's core capabilities. It provides a high-level view of system boundaries, showcasing user privileges ranging from dashboard management to triggering the 2D-to-3D generative pipeline.

Figure 3.4: Eventvista System Activity Diagram



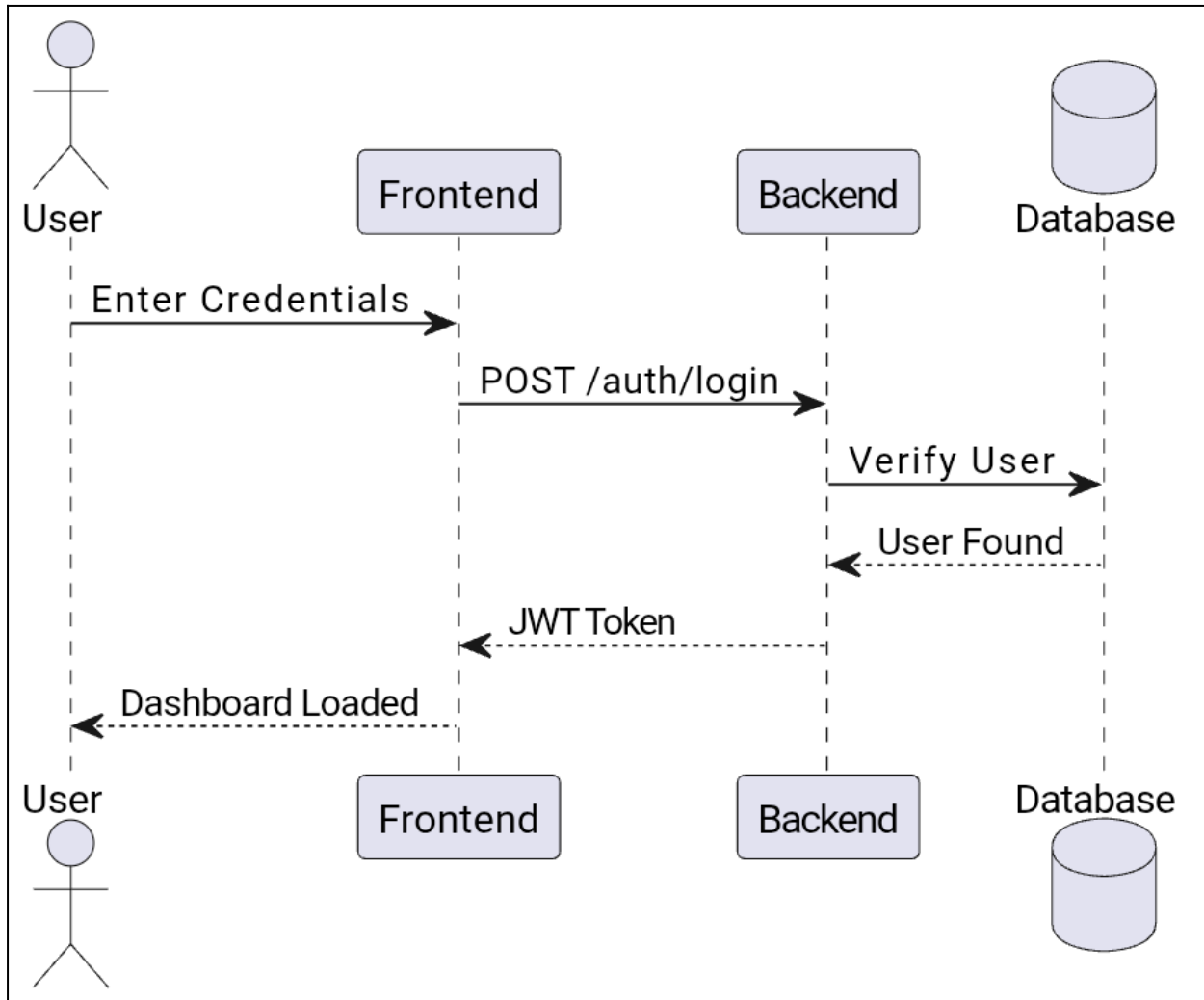
This diagram models the application's operational step-by-step workflow. It charts the logical decisions and paths taken by the system from the moment a user signs in, through generating custom layouts and manipulating 3D canvases, to checking out via payment gateways.

Figure 3.5: Eventvista Object-Oriented Class Diagram



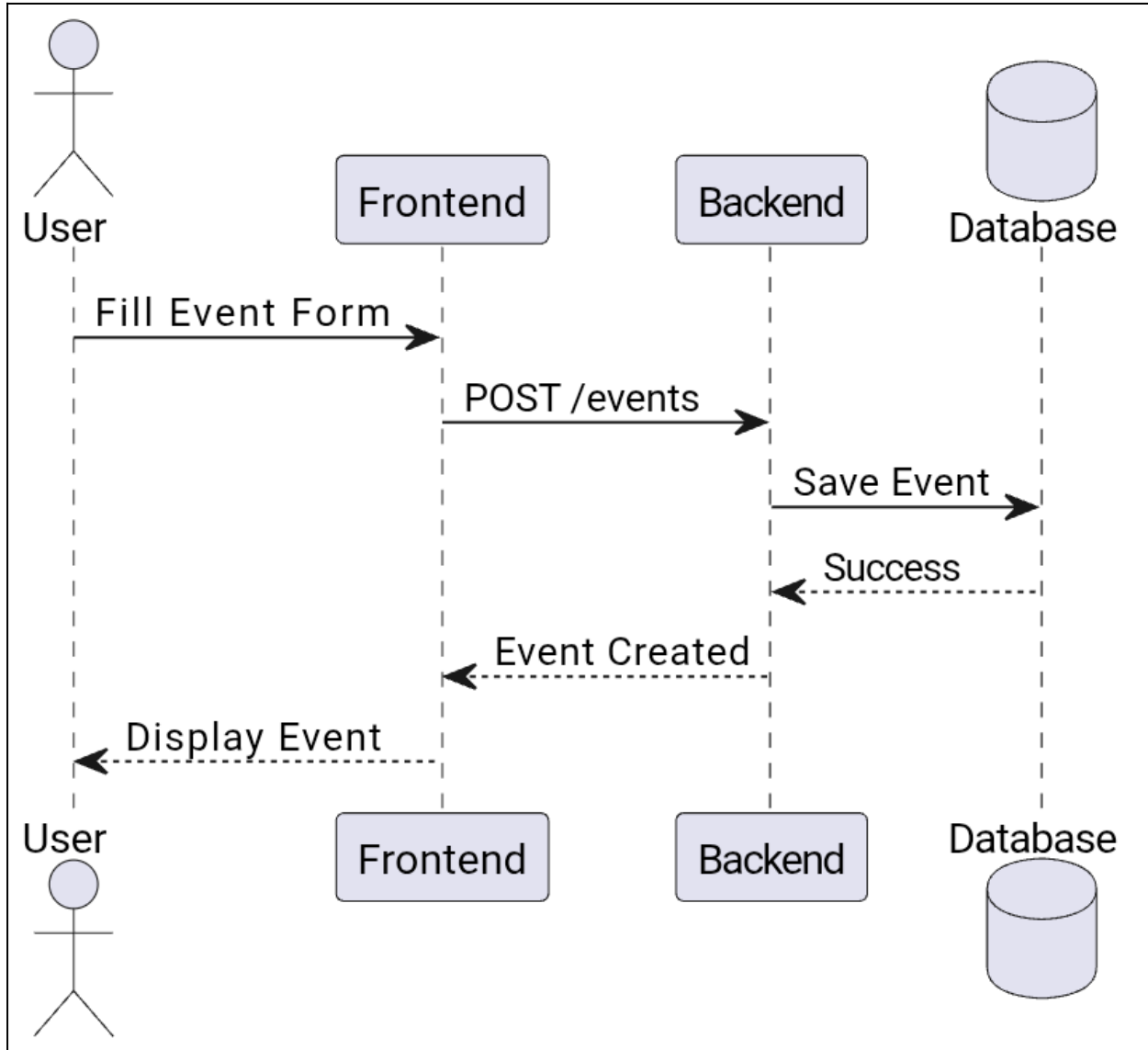
This diagram details the static code architecture and object-oriented structure of Eventvista. It explicitly outlines the internal attributes, methods, and structural relationships (such as inheritance and associations) between classes like Users, Events, Venues, and Canvas Objects.

Figure 3.6: Sequence Diagram for User Login and Authentication



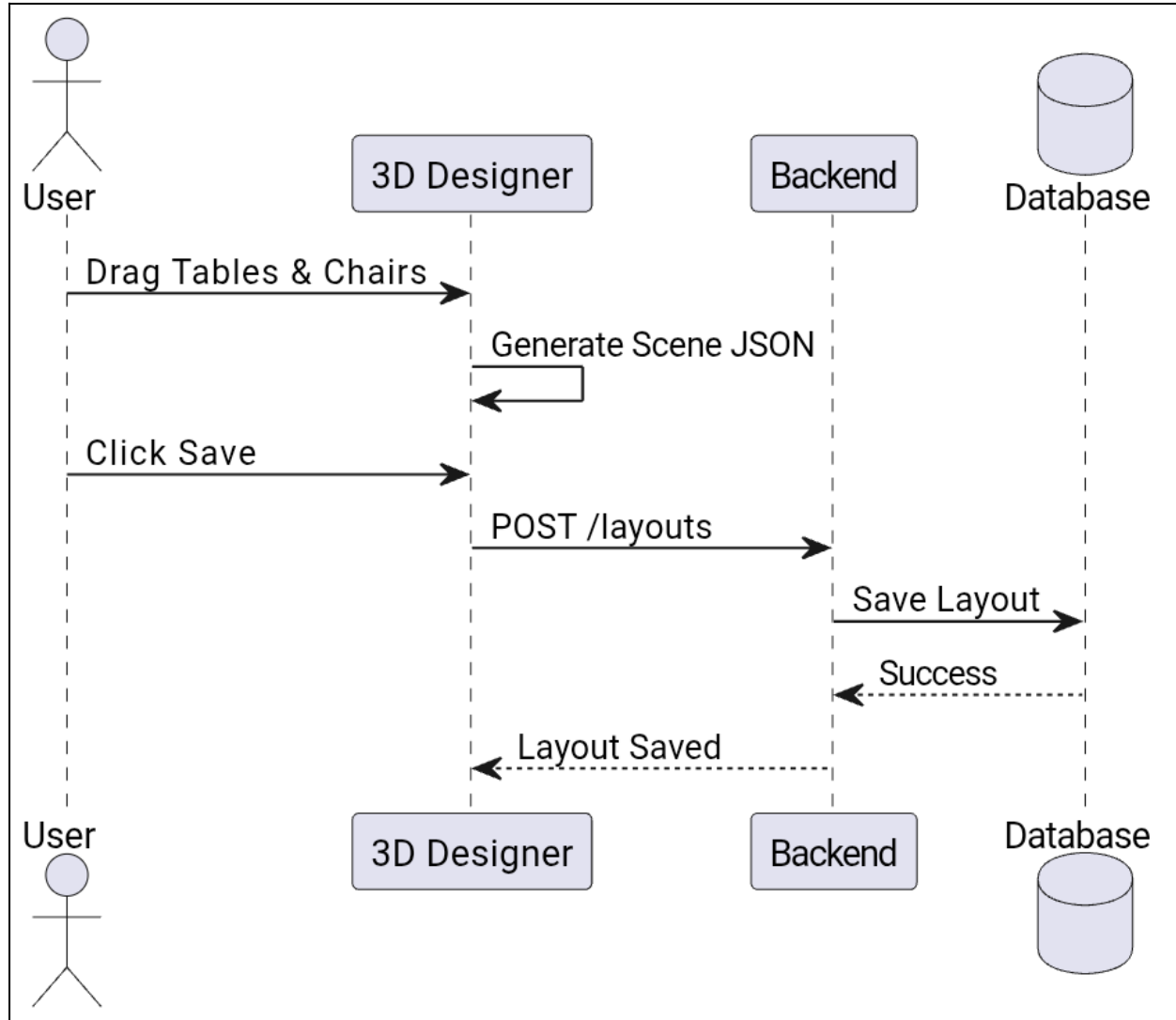
This diagram shows the exact sequence of messages exchanged during user login. It displays the step-by-step communication between the browser UI, the Node.js Authentication Controller, and the database grid to verify user credentials securely.

Figure 3.7: Sequence Diagram for Event Creation & AI Parameter Generation



This diagram illustrates the dynamic process of launching a new event. It shows how user inputs are passed from the web UI to the Event Controller, which then calls the Groq LPU engine to instantly calculate predictive spatial parameters before storing the event details.

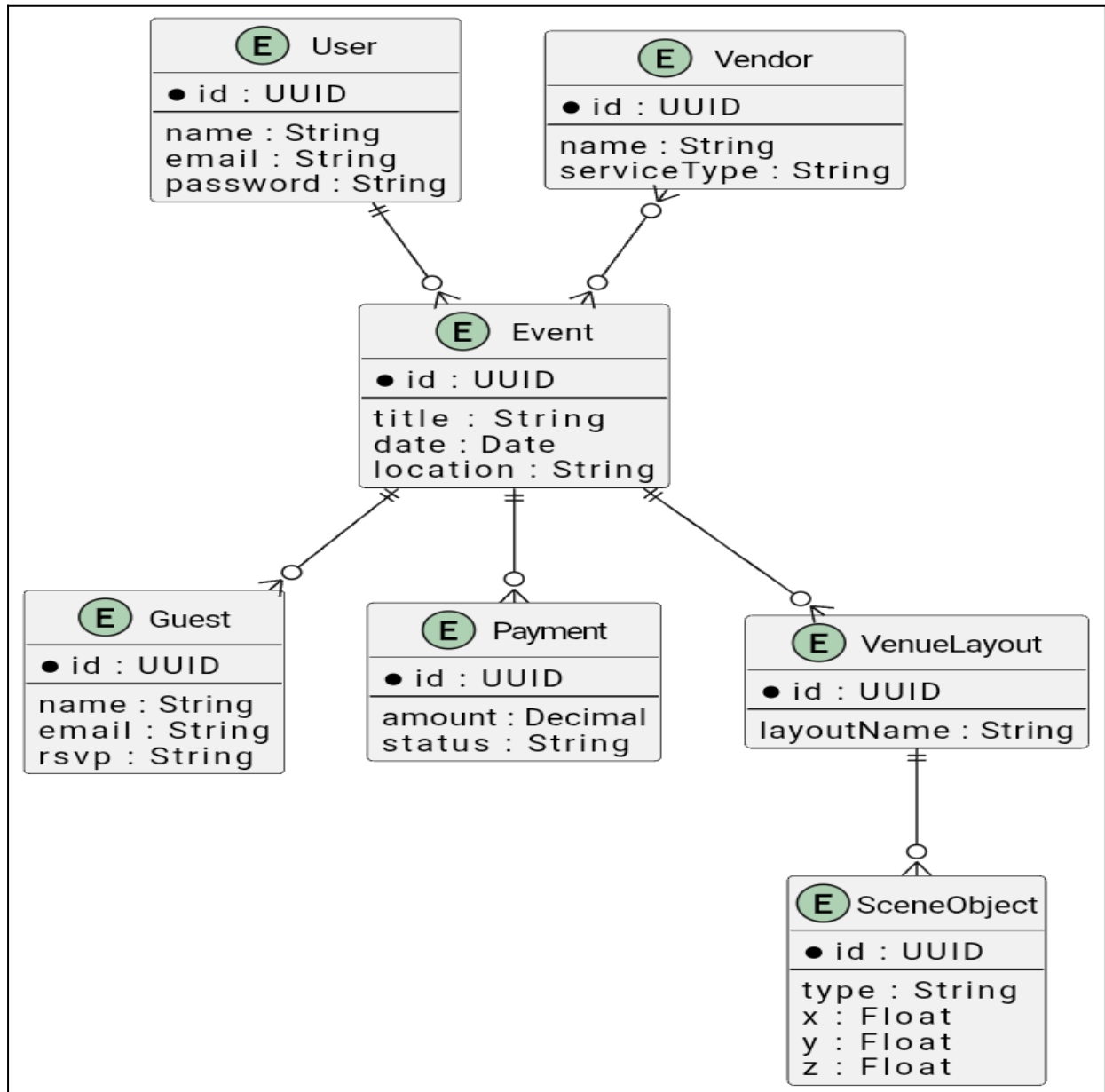
Figure 3.8: Sequence Diagram for Saving a 3D Venue Layout



This diagram tracks the backend process triggered when a user clicks the "Save Layout" button. It shows how interactive item coordinates from the React Three Fibre canvas are packaged as JSON, sent via API endpoints, and successfully written to the database.

3.3 Database Entity-Relationship Diagram (ERD)

Figure 3.9: Eventvista Entity-Relationship Diagram (ERD)



This diagram illustrates the data schema and relationship boundaries governing the Eventvista ecosystem. It shows the tables or collections (Users, Events, Guests, Vendors, and Payments), highlighting primary and foreign keys that guarantee database integrity and smooth data flow.

3.4 Core Technology Stack Details

User UI Layer: Next.js framework compiled with Tailwind CSS, deployed on Vercel for global edge network delivery, fast continuous integration (CI/CD), and optimal React Three Fiber (R3F) rendering.

Core Logic Layer: Node.js backend managing an Express.js router structure, deployed on Render as a Web Service to ensure seamless API connections and straightforward GitHub deployment hooks.

Database Grid: MongoDB for flexible schema modelling of variable spatial floor arrangements, combined with PostgreSQL to secure financial processing, user metadata registries, and transaction accounts.

Computational Framework Interfaces: Groq API: Interfaced using the ultra-low latency `llama-3.1-70b-versatile` system model to structure spatial configuration parameters instantly.

TRELLIS 3D Pipeline: Open-source generative framework (`TRELLIS.2-4B`) operating through isolated GPU endpoints on Hugging Face Spaces to generate highly accurate GLTF spatial meshes from 2D venue images.

Payment Orchestration Network: Local mobile-first transaction clearing managed natively through the Mpesa API Gateway.

Document Generation & Approval Stack:

MeshLab: An open-source backend tool programmed to automatically convert the raw 3D venue models generated by the AI into the U3D format, which is required for PDF embedding.

LaTeX: An open-source typesetting system that will pull event specifications (such as audience size, budget, and venue details) from the database, format them into a professional document, and embed the interactive U3D model directly into the centre of the page.

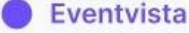
OpenSign: A free, open-source digital signature platform. Once the PDF is generated, OpenSign will securely email it to the client and provide a legally binding electronic signature slot for final approval.

4. FRONTEND INTERFACE REQUIREMENTS

4.1. User Interface Wireframes

Figure 1: Public Landing Page & Onboarding Entry

Login and authentication screen



Plan. Manage. Celebrate.

Eventvista helps you organize events seamlessly and create unforgettable experiences



Welcome Back!

Sign in to continue to your account

Email Address

Password

☐ Remember Me [Forgot Password?](#)

LOGIN

OR

 **Continue with Google**

[Don't have an account?](#) [Sign Up](#)

2024 Eventvista. All rights reserved.

This interface serves as the primary public gateway for unregistered visitors and prospective clients. It features a clean global navigation bar providing instant access to platform capabilities, pricing models, and company information. The central hero section delivers a clear value proposition highlighting 'Eventvista's AI-powered spatial design features, supported by an intuitive 'Get Started' call-to-action button that routes users directly into the onboarding pipeline.

Figure 2: User Sign-In & Authentication Interface

User Registration Interface



Let's Get Started!

Create an account and start planning amazing events with Eventvista



Create Your Account

Fill in the details below to get started

Full Name

Email Address

Phone Number

Password

Confirm Password

Role

Client

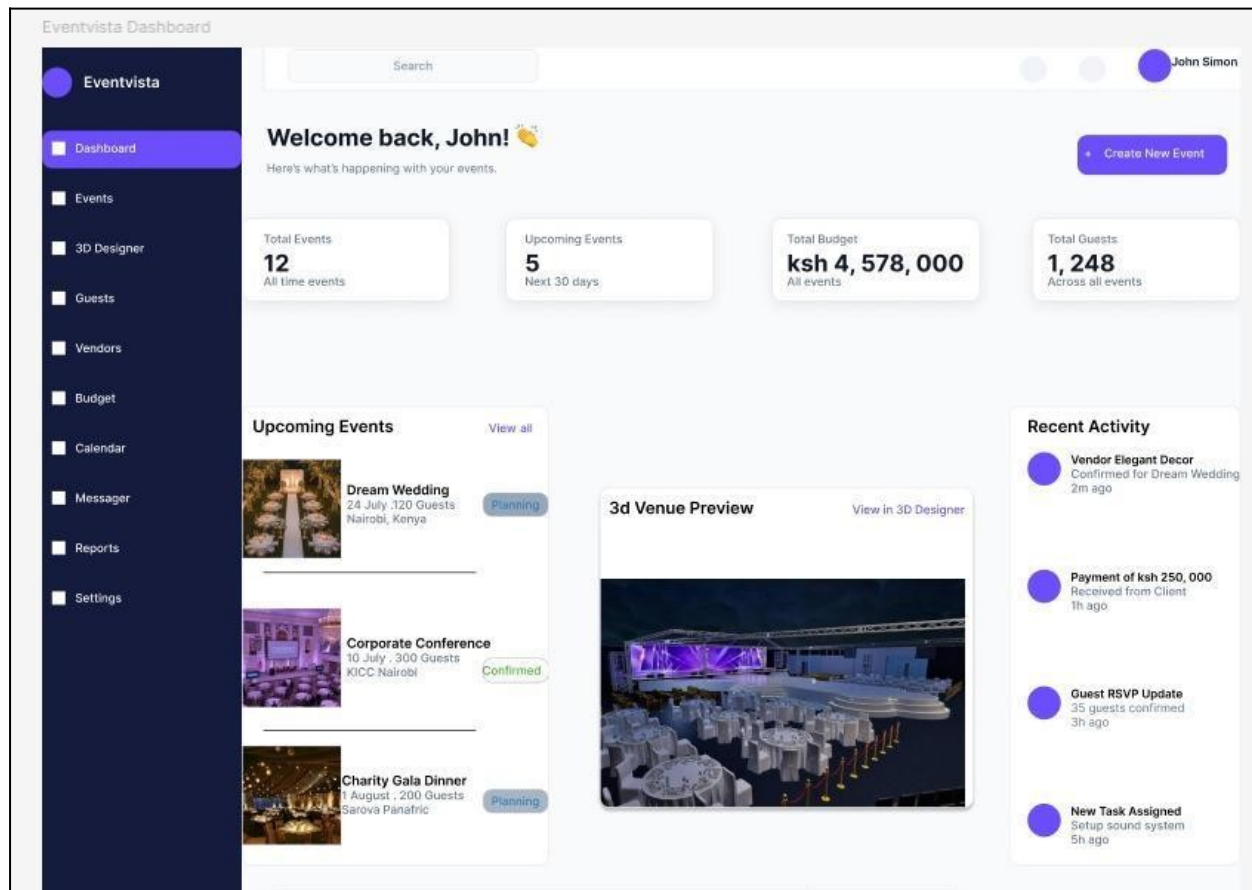
CREATE ACCOUNT

[Already have an account?](#) [Login](#)

2024 Eventvista. All rights reserved.

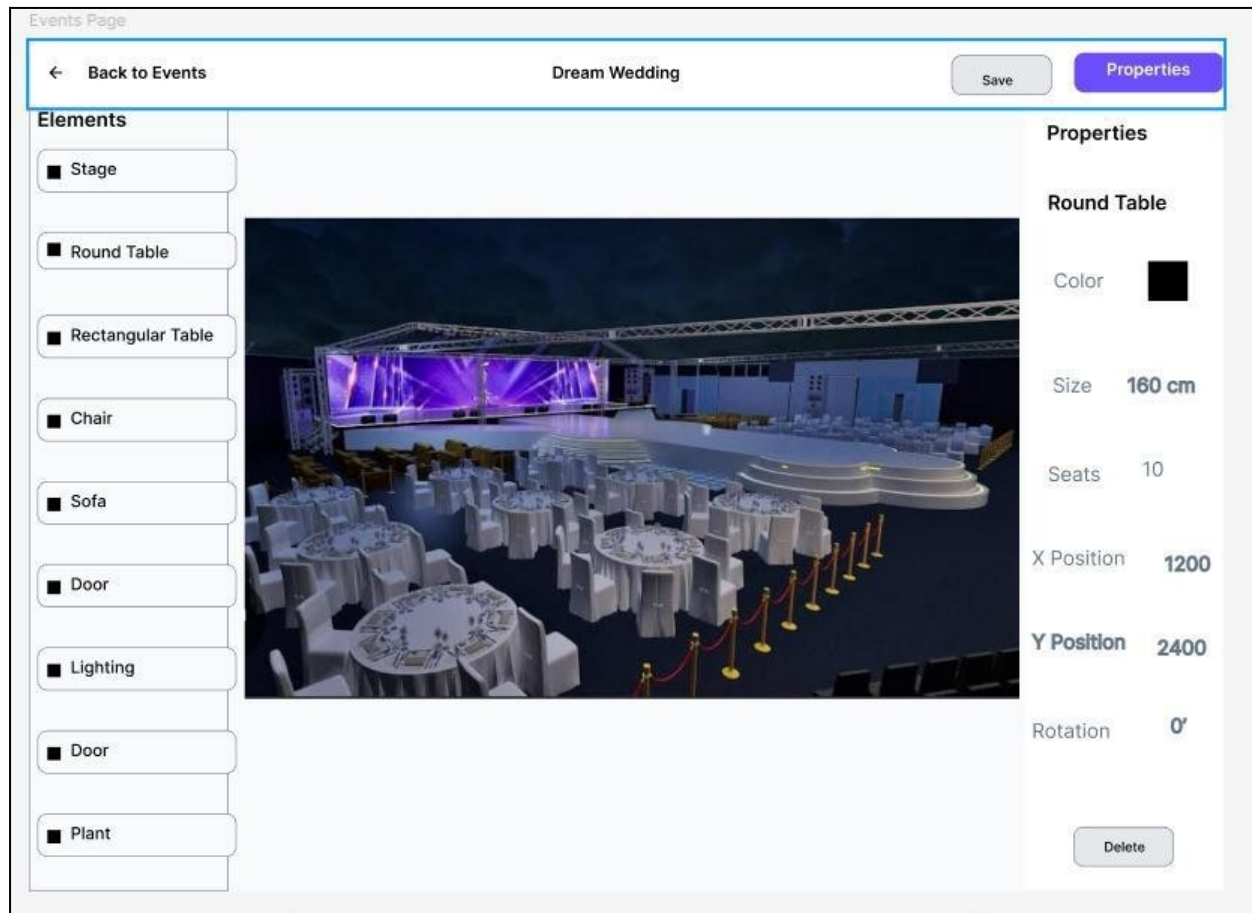
A secure, minimalist interface designed for returning users to safely access their private accounts. The layout provides dedicated input text boxes for email addresses and account passwords, complete with integrated real-time data validation. It incorporates standard navigational aids, including a 'Remember Me' checkbox, a secure 'Forgot Password?' credential recovery link, and alternative single sign-on (SSO) options using trusted federated identity providers.

Figure 3: Event Dashboard & Overview



This dashboard acts as the command centre for the user. It provides a high-level overview of current and upcoming events, allowing users to track progress, monitor budgets, and access active projects at a glance.

Figure 4. 3D Venue Planning Workspace



The core feature of Eventvista, this workspace provides a drag-and-drop environment where users can visually arrange tables, chairs, stages, and lighting. It translates simple floor dimensions into a realistic 3D preview, allowing for precise spatial planning.

Figure 5. Event Management Module

Events Page

Eventvista

Events

Dashboard

3D Designer

Guests

Vendors

Budget

Calendar

Messenger

Reports

Settings

Events

+ New Event

Search Events

All Status

Event Name	Date	Venue	Guests	Status	Action
 Dream Wedding	24 July 2026	Nairobi, Kenya	120	Planning	+++
 Corporate Conference	10 July 2026	KICC, Nairobi	300	Confirmed	+++
 Charity Gala Dinner	1 August 2026	Sarova Panafric	200	Planning	+++
 Product Launch	17 August 2026	TRM, Nairobi	150	Confirmed	+++
 Birthday Party	30 August 2026	Karen Club	80	Planning	+++

A detailed interface allowing users to create, edit, and manage individual event details. It acts as the primary control panel for scheduling and operational logistics for each event.

Figure 6: Guest List Management

Guests Page

Guests

Import

+ Add Guest

Search Guests

RSVP

Name	Email	Phone	RSVP Status
Alice Johnson	alice@gmail.com	0712242544	Confirmed
Brian Smith	brian@gmail.com	0700345675	Pending
Catherine Lee	catherine@gmail.com	0754757004	Confirmed
David Brown	david@gmail.com	0111223354	Declined
Emma Wilson	emma@gmail.com	0711767134	Pending

A functional table-based interface for managing the guest list. It enables organisers to track attendance status, add guest contact details.

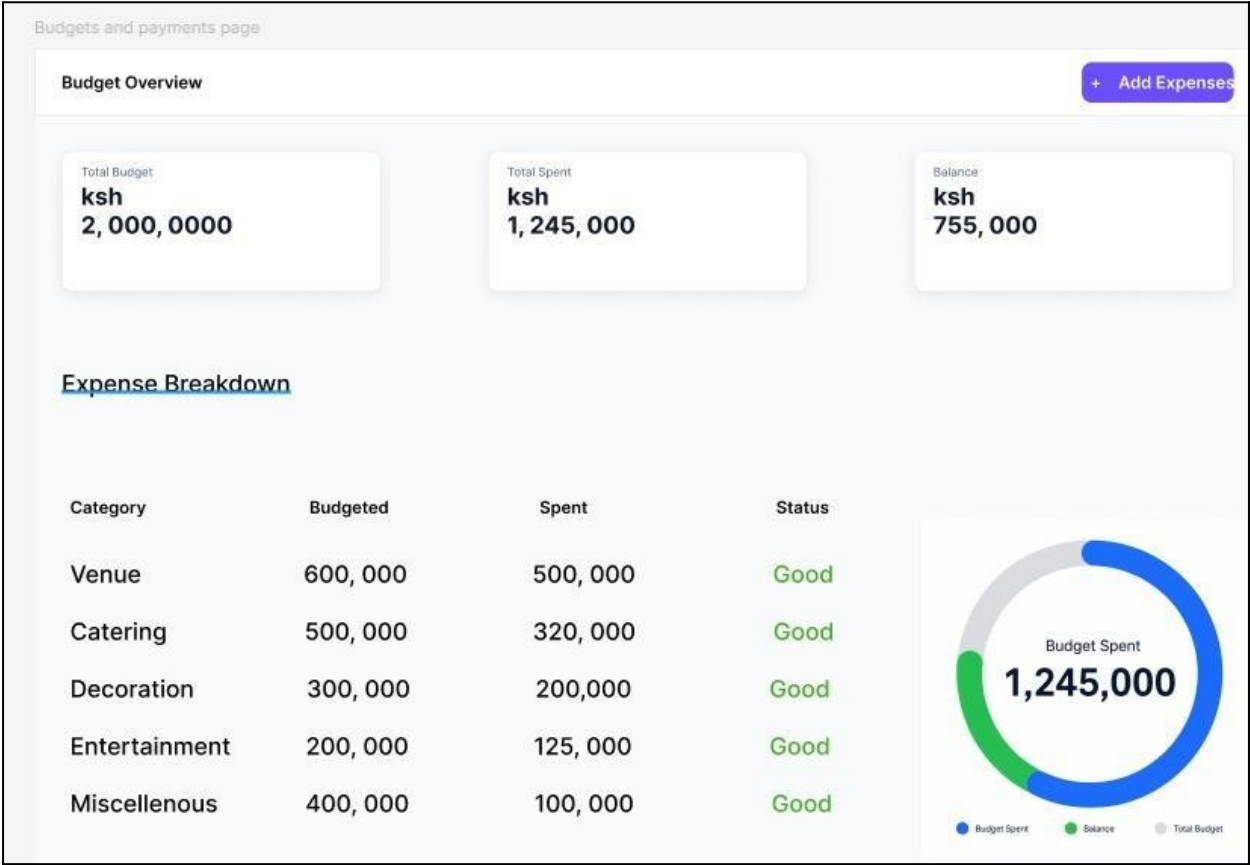
Figure 7. Vendor List Management

Vendors Page

Vendors					+ Add Vendor
Search vendors...			All Categories ▼		
Vendor	Category	Contact	Status		
 Elegant Decor	alice@gmail.com	0712242544	Confirmed		
 Classic Catering	brian@gmail.com	0700345675	Pending		
 Perfect Sounds	catherine@gmail.com	0754757004	Confirmed		
 Bright Lights	david@gmail.com	0111223354	Declined		
 Royal Photography	emma@gmail.com	0711767134	Pending		

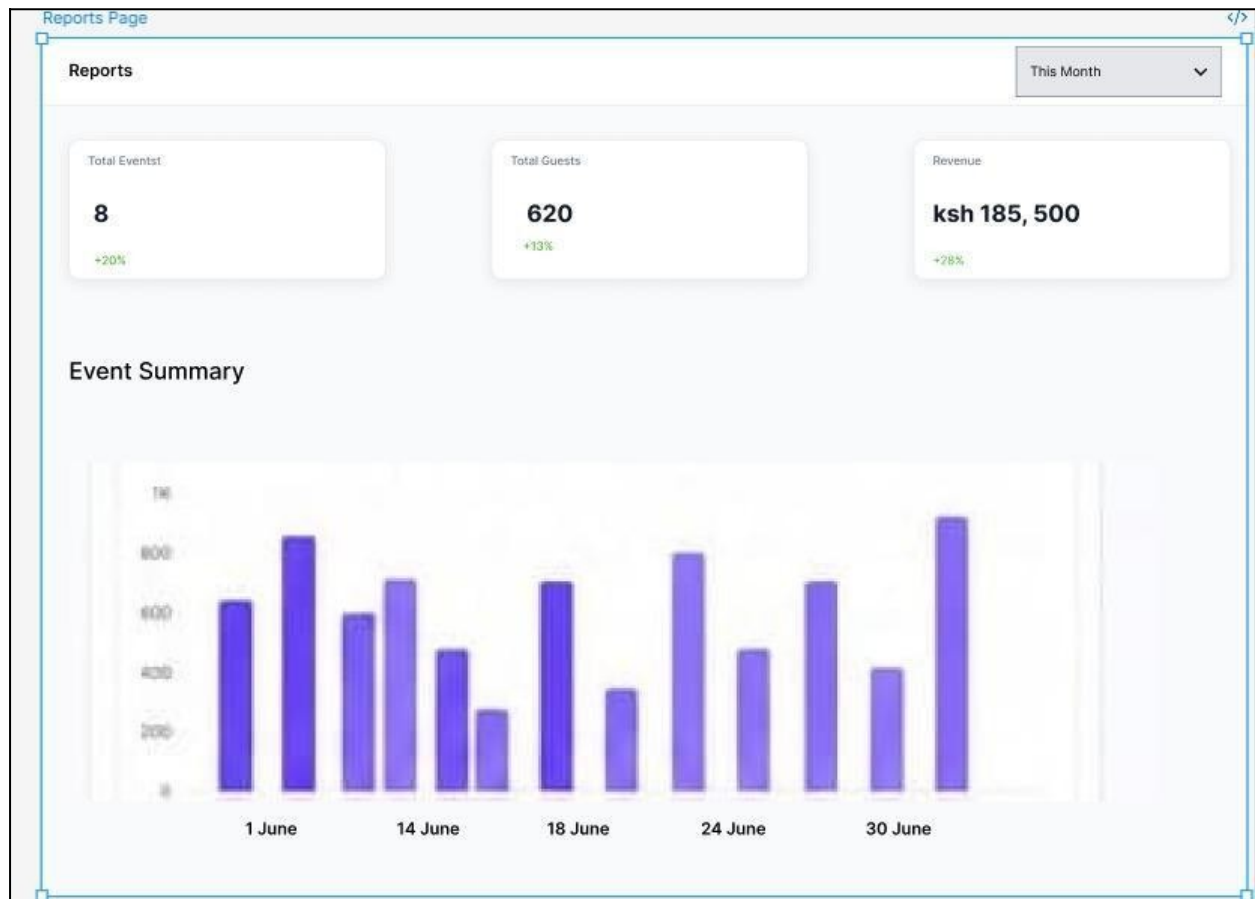
This section enables users to manage their inventory and select service providers. It features a simplified interface to assign vendors to specific roles, such as catering, sound, and décor, directly within the event timeline.

Figure 8. Budgeting and payments page



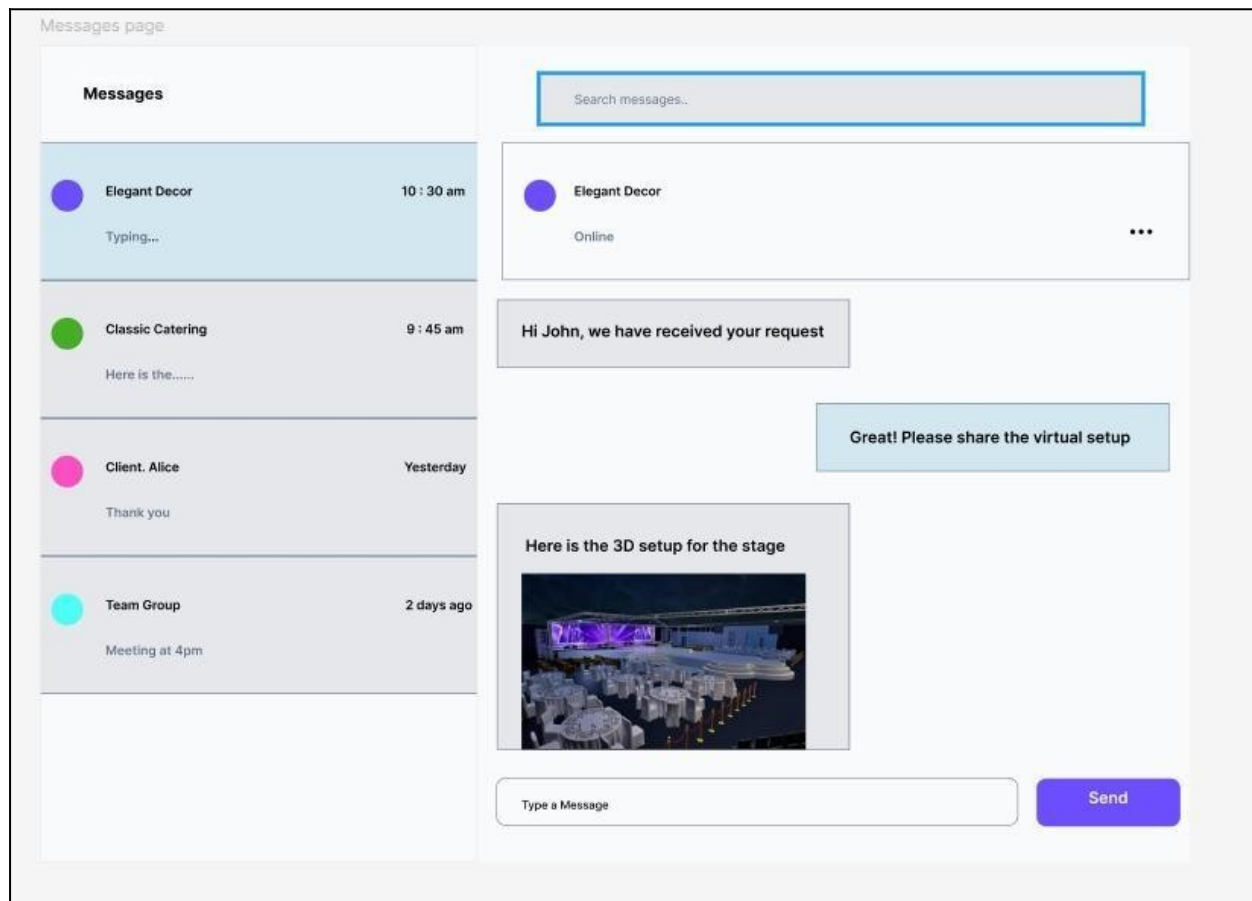
A dedicated financial module that displays payment records, budget allocation, and expenditure tracking. It provides the organiser with real-time insights into the financial health of the event.

Figure 9. Reports and Analytics Dashboard



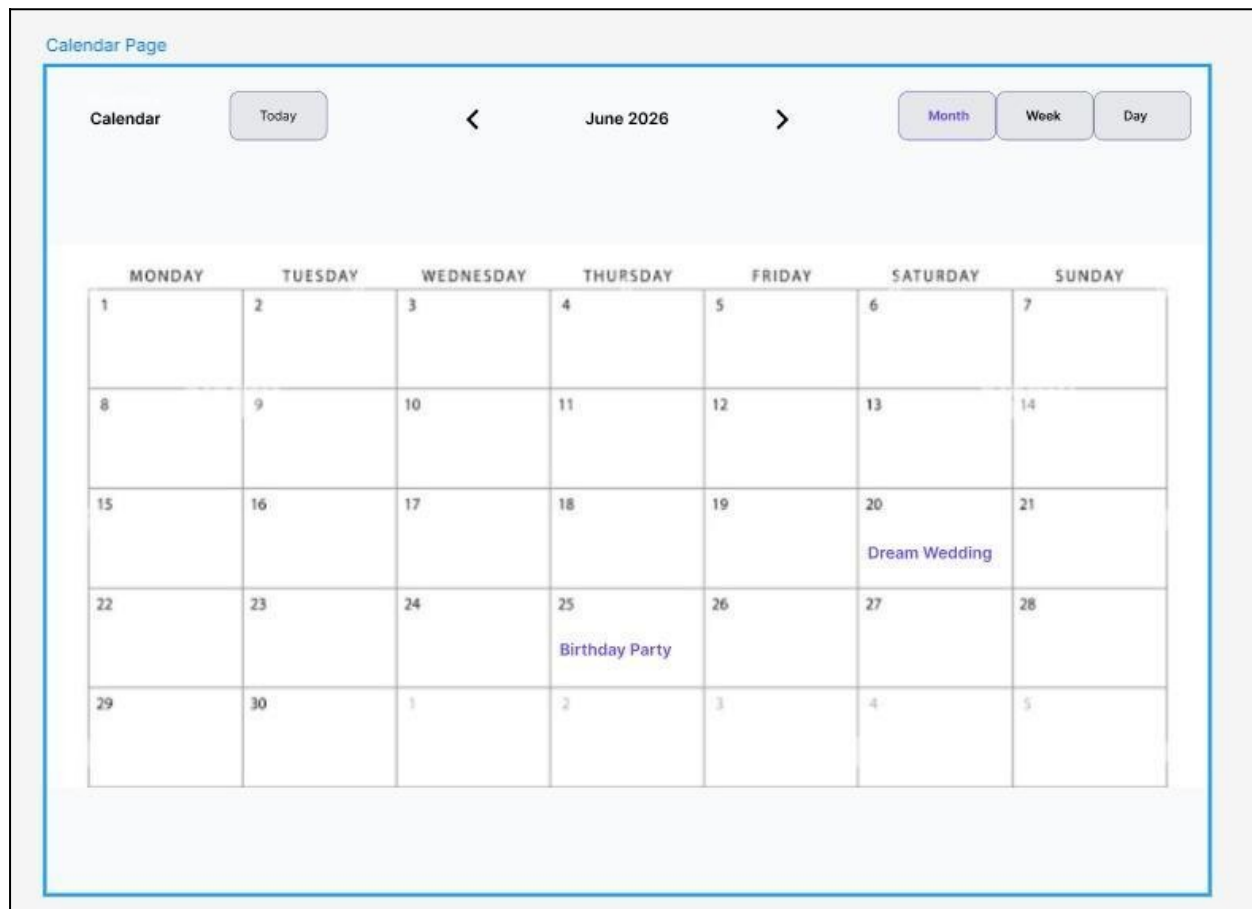
An analytical overview interface designed to compile and display comprehensive real-time and post-event data. It provides visual data charts, budget variance reports, and vendor rating statistics, allowing organisers to seamlessly export performance insights into PDF or spreadsheet formats for stakeholder review.

Figure 10. Centralised Messaging and Collaboration Hub



The dedicated communication gateway within the platform that bridges the gap between organisers, clients, and external vendors. This wireframe presents a clean multi-pane layout featuring text chat, chronological conversation threads grouped by specific event tasks, and real-time status notifications to streamline coordination.

Figure 11: Interactive Event Calendar and Scheduler



A master timeline planner offering customisable monthly, weekly, and daily views. It maps all time-sensitive milestones across multiple events, such as venue walkthroughs, vendor setup appointments, final guest RSVP cutoff dates, and pending payment deadlines, ensuring no critical operational task is overlooked.

4.2 Hardware Interfaces

To ensure a seamless user experience, particularly when interacting with the 3D venue designer, the following hardware requirements apply. These ensure that WebGL and React Three Fiber canvases render with sufficient stability and frame rates:

Graphics Processing Unit (GPU): A dedicated or integrated GPU capable of supporting WebGL 2.0 is required. Modern hardware, such as that found in standard laptops or tablets released within the last 5 years, is generally sufficient for real-time 3D rendering.

Memory (RAM): A minimum of 8GB of system RAM is recommended to support smooth browser performance while maintaining active AI model processes and 3D asset manipulation.

Display Resolution: A minimum screen resolution of 1366x768 pixels is required to interact effectively with the drag-and-drop toolbars and canvas controls without user interface overlap.

Input Devices: A pointer device (mouse or trackpad) is highly recommended over touch-only interfaces, as the 3D venue design tool requires precise clicking, dragging, and rotation control for spatial planning.

4.3 Software Interfaces

Eventvista utilises a modern decoupled software architecture to ensure scalability and ease of maintenance. The interfaces between the frontend and backend are managed as follows:

Frontend (Next.js): The user interface is built using Next.js. It interacts with the backend solely through defined API endpoints. The frontend is responsible for state management, user authentication flows, and rendering the 3D interactive components.

Backend (Node.js/Express): The backend serves as the core API gateway. It exposes RESTful API routes that the frontend calls to perform CRUD (Create, Read, Update, Delete) operations. This includes retrieving event data, processing budget calculations, and saving 3D layout configurations.

Data Exchange Format: All communication between the frontend and backend is conducted using standard JSON (JavaScript Object Notation), ensuring consistent data parsing and structure across the entire application stack.

4.4 Communications Interfaces

To maintain high standards of security and real-time interaction, Eventvista employs the following communication protocols:

Secure Transport (HTTPS): All data transmitted between the client browser and the server is encrypted using Transport Layer Security (TLS/HTTPS). This protects sensitive information, such as payment details and user credentials, from interception during transmission.

Real-time Communication (WebSockets): The messaging and collaboration hub uses WebSockets to establish a persistent, bidirectional communication channel between the user and the platform. This allows for instantaneous updates, such as incoming messages or notification alerts, without requiring the user to refresh the page.

RESTful Protocol: The primary interface for general application logic follows RESTful design principles. This uses standard HTTP methods (GET, POST, PUT, DELETE) to interact with resources, ensuring a clean, predictable, and stateless interface between the client and the server.

5. FUNCTIONAL REQUIREMENTS

5.1 Frontend Interface Capabilities (Phase 7 Integration)

FR-1.1: The frontend, deployed via Vercel, must render modular UI components, including Navigation, Sidebars, analytical Dashboards, Data Tables, Input Forms, Charts, Interactive Calendars, and User Profile Settings, based on the Figma wireframes.

5.2 Backend API Processing (Phase 8 Integration)

FR-2.1: The Render-hosted backend must independently operate and resolve queries for the following core endpoints: Authentication API, Users API, Events API, Guests API, Vendors API, Payments API, Reports API, and Notifications API. Test-driven verification must be applied to each before integration.

5.3 3D Venue Designer Engine (Phase 10 Integration)

FR-3.1: The system must feature a unique interactive canvas allowing users to: drag tables and chairs, move staging areas, add decor (flowers, lighting), position DJ booths, save layouts to the database, and preview the venue in full 3D.

5.2 Interactive Event Configuration Setup (CRUD Operations)

FR-2.1: Users must be able to perform standard CRUD operations on all events (create name, modify timelines, add logistical markers, delete entries).

FR-2.2: The application must maintain an active asset tracking array linking equipment, decor, staging elements, and sound structures dynamically inside a centralised cloud profile.

5.3 2D-to-3D Visual Pipeline Processing

FR-3.1: The system must accept standard mobile 2D image uploads (`PNG`, `JPEG`) of physical venues.

FR-3.2: The system must extract structural data from the image and feed it to the TRELLIS pipeline on Hugging Face to generate a 3D structural canvas mesh (`.gltf` / `.glb`) within 30 seconds of submission.

FR-3.3: The system must allow real-time browser modification (scaling, rotation, spatial displacement) of the generated furniture and structural elements via React Three Fiber.

Final Event Plan Export and Client Approval Workflow:

1. Once the event organiser is satisfied with the AI-generated 3D venue and the budget parameters, they click the 'Generate Final Plan' button.
2. The system works in the background to automatically compile a single PDF containing the interactive 3D model alongside the formatted text of the event specifications.
3. The system immediately emails a secure OpenSign link to the client.
4. The client clicks the link to review the event details, interacts with the 3D venue model within their web browser, and digitally signs the document to give official approval.

5.4 Groq-Driven Predictive Analytics Engine

FR-4.1: The backend must compile user input metrics (total guest budget, absolute target headcounts, asset requests) into a structured payload for the Groq API.

FR-4.2: The system must parse Groq's high-speed JSON responses to display real-time warnings if the event's structural density compromises fire exits or breaks budget ceilings.

5.5 Guest Management, Ticket Processing, and Automated Communications

FR-5.1: The software must provide functional landing interfaces for online guest registration, seat reservation, and attendee confirmation tracking.

FR-5.2: The communication grid must generate automated reminder alerts across dedicated channels (Email, SMS notifications, WhatsApp API hooks) when event schedules shift.

6. NON-FUNCTIONAL REQUIREMENTS

6.1 Performance and Speed Requirements

NFR-1.1: Text configurations, budget table modifications, and analytical checks evaluated by the Groq API must process and display on the frontend within 800 milliseconds.

NFR-1.2: The 3D viewport engine must maintain a rendering refresh pace of at least 60 Frames Per Second (FPS) during interactive camera pans across mid-tier mobile consumer hardware devices.

6.2 Data Security and Encryption

NFR-2.1: Financial processing channels must integrate absolute cryptographic protocols matching local network processing requirements.

NFR-2.2: User database structures must obfuscate passwords using industry-standard hashing algorithms (e.g., bcrypt with a high work factor).

6.3 Scalability and Availability

NFR-3.1: The frontend architecture deployed on 'Vercel's Edge Network must automatically cache dynamic routes and autoscale to handle high-traffic surges during peak event registration times.

NFR-3.2: The backend logic managed on Render must guarantee continuous deployment from the main Git repository and maintain a minimum core operational availability benchmark of 99.9% annually.

7. Project Management & Analysis Models

7.1 Project Implementation Plan & Methodology (Agile Framework)

For the Eventvista project, we utilise an Agile Framework. Unlike traditional linear (Waterfall) models where you finish one stage before starting the next, Agile is iterative and flexible. It allows the team to develop the software in small, manageable cycles (often called "sprints").

Why we chose Agile for Eventvista:

Flexibility: As we develop complex AI-driven spatial planning tools, requirements may evolve based on user testing. Agile allows us to adapt to these changes without derailing the entire project.

Continuous Feedback: We involve stakeholders and users throughout the process, ensuring the final output actually solves the problem of inefficient event planning.

Incremental Delivery: By breaking the 12 phases into smaller chunks, we can test and refine features (like the 3D venue designer) early, rather than waiting until the end to discover issues.

7.2 Core Work Breakdown Structure (WBS) Tasks

The WBS is our "project roadmap." It breaks the large Eventvista project into 12 distinct, manageable phases. This ensures that every team member knows their specific responsibilities and how their work contributes to the finished product.

Phase 1: Inception, Requirements Analysis & Logic Mapping [Gideon & Lewis]

- |— 1.1 Conduct target user persona research (Event Organisers, Vendors, Guests).
- |— 1.2 Define system boundary matrix and functional requirements.
- └— 1.3 Draft and finalise SRS documentation.

Phase 2: System Architecture & Logic Design [John & Gideon]

- |— 2.1 Design relational database schemas (Users, Events, Vendors, Payments).
- |— 2.2 Map backend logic and RESTful API endpoints.
- └— 2.3 Architect the data pipeline connecting the Groq API and TRELLIS 3D model.

Phase 3: UI/UX Prototyping & Interface Design [Shekinah]

- |— 3.1 Create low-fidelity wireframes for the user journey.
- |— 3.2 Design high-fidelity interactive mockups in Figma.
- └— 3.3 Finalise accessible Tailwind CSS layout controls and colour systems.

Phase 4: Infrastructure & Environment Setup [Lewis & John]

- |— 4.1 Initialise Git repositories and version control workflows.
- |— 4.2 Provision Vercel (Frontend) and Render (Backend) hosting environments.
- └— 4.3 Configure secure environment variables and API keys (Groq, Hugging Face).

Phase 5 and 6 : [Phase skipped/merged in SDLC - proceeding to build phases]

Phase 7: Build the Frontend [Shekinah & John]

- |— 7.1 Translate Figma designs into Next.js structural components.
- └— 7.2 Build core UI features: Navigation, Sidebar, Dashboard, Cards, Tables, Forms, Charts, Calendar, Profile, and Settings.

Phase 8: Build the Backend [John]

- |— 8.1 Establish the Node.js / Express.js API Gateway.
- └— 8.2 Create and test core APIs: Authentication, Users, Events, Guests, Vendors, Payments, Reports, and Notifications.

Phase 9: Connect Everything (System Integration) [John]

- |— 9.1 Link the Vercel Frontend to the Render Backend API endpoints.
- └— 9.2 Ensure seamless CRUD data flow between the UI and the database.

Phase 10: Build the 3D Venue Designer (Core Feature) [John & Shekinah]

- |— 10.1 Integrate React Three Fiber (R3F) for browser-based 3D rendering.
- |— 10.2 Implement drag-and-drop mechanics (tables, chairs, stage, lighting, DJ booth, flowers).
- └— 10.3 Enable saving, fetching, and previewing of full 3D layouts via the database.

Phase 11: Testing & QA Validation [Lewis, Gideon & Team]

- |— 11.1 Execute Unit and Integration testing on Login and Registration.
- |— 11.2 Verify Event saving, Guest routing, Vendor assignment, and Payment recording.
- └— 11.3 Test 3D designer load times, collision detection, and layout saving integrity.

Phase 12: Final Deployment & Handover [Lewis]

- |— 12.1 Publish Production branches (Frontend to Vercel, Backend/DB to Render).
- └— 12.2 Present final system and SRS Capstone documentation for grading.

By following this structured approach, the team ensures that Eventvista is built with academic rigour while maintaining the flexibility required for successful modern software delivery.

8. LEGAL AND ETHICAL CONSIDERATIONS

Data Protection Compliance: Absolute structural alignment with international standards and the Kenya Data Protection Act of 2019. Venue spatial pictures and user contact metrics will be strictly isolated and encrypted using TLS 1.3.

Ethical Safety Guards: Spatial generation scripts will incorporate automatic alerts preventing modifications that knowingly compromise regional safety rules, indoor escape route corridors, or legal fire capacity benchmarks.

8.2 Ethical and Explainable Use of AI

Eventvista relies heavily on Artificial Intelligence to convert 2D images into 3D models and to automate budget forecasting. It is a core ethical requirement that this AI operates transparently. "Explainable AI" means that the system does not act as a mysterious 'black box'. When Eventvista's AI restricts a user from placing too many tables in a 3D canvas, or when it calculates a specific budget estimate, it will provide clear, readable prompts explaining why (e.g., "This layout exceeds the safe fire-exit capacity for this room size"). Furthermore, our AI models are programmed to remain neutral and unbiased, ensuring that all venue designs and vendor recommendations are generated fairly and without prejudice.

8.3 Oversight Bodies

To ensure compliance with legal and ethical standards, Eventvista's development and operations adhere to the guidelines set by relevant oversight authorities. As the platform handles personal user information and M-Pesa payment details, it must align with the regulations of the Office of the Data Protection Commissioner (ODPC) in Kenya. Additionally, because this is a capstone project developed at The Technical University of Kenya, the system's design and research methodology are subject to the oversight of the 'university's Academic and Ethical Review Boards. This ensures the project maintains high academic integrity and responsible software engineering practices.

9. CONCLUSION AND FUTURE RECOMMENDATIONS

9.2 Future Iterations and Recommendations

While the current version of Eventvista successfully delivers an interactive 3D venue designer and automated budgeting, there is significant room for future growth. For future iterations, we recommend the following enhancements:

Virtual Reality (VR) Integration: Allowing event organisers and clients to put on a VR headset and physically "walk through" the AI-generated 3D venue before making final decisions.

Live Vendor Bidding System: Upgrading the vendor module so that local caterers, DJs, and decorators can view a user's 3D event canvas and submit competitive price quotes directly through the platform.

Paperless Vendor Contracting: Expanding the OpenSign integration beyond client approvals so that when local vendors (such as caterers or DJs) win a bid on the Live Vendor Bidding System, the system automatically generates and sends them a secure, legally binding digital contract to sign.

Real-time AI Decor Suggestions: Training the AI to automatically suggest colour schemes and decor placements based on the specific cultural or thematic requirements of the event.

9.3 Project Potential for Research and Teaching

Eventvista serves as a highly practical case study for academic institutions. Within the fields of Information Science, Software Engineering, and Infopreneurship, this project demonstrates exactly how to integrate modern AI APIs (such as Groq and Hugging Face) with traditional web development to solve real-world business problems. The 'project's architecture, system logic, and complete source code can be utilised as a teaching tool by university lecturers. It provides future students with a tangible example of how to manage a full Software Development Life Cycle (SDLC) while building commercially viable, AI-driven software.